

**REPORT
OF
SMS SPAM DETECTION**

**SUBMITTED
TO
SCHOOL OF COMPUTER SCIENCE AND
ARTIFICIAL INTELLIGENCE**

IILM UNIVERSITY, GREATER NOIDA

**IN PARTIAL FULFILLMENT OF THE REQUIREMENT OF
DEGREE OF**

B.TECH CSE (AI & ML)



By

NAME OF STUDENT : TARUN KUMAR YADAV

ROLL NUMBER : 25SCS1003003532

BRANCH : AI ML

2026-27

CANDIDATE'S DECLARATION

I, TARUN KUMAR YADAV do hereby declare that the internship report titled "**SMS SPAM DETECTION**" has been completed by me to fulfil the requirement for the award of the degree of Bachelor of Technology in CSE with specialization in AI ML. All the references have been quoted and I have not taken as such any material from any other source. I affirm to you that this report is my own and has not been submitted to any other institute for any degree/diploma requirement and further I shall be solely responsible for any kind of copyright violation in this regard.

Signature



Student Name : TARUN KUMAR YADAV

Roll No. : 25SCS1003003532

ACKNOWLEDGEMENT

I am using this opportunity to express my gratitude to everyone who supported me throughout the Internship Program. I am thankful for their inspiring guidance, invaluable constructive criticism and friendly advice during this work. I am sincerely grateful to them for sharing their truthful and illuminating views on a number of issues related to this work. I express my gratitude to my parents for their blessings.

Signature

A handwritten signature in black ink that reads "Tarun Yadav". The signature is written in a cursive style with a horizontal line underneath the name.

Student Name : TARUN KUMAR YADAV

Roll No. : 25SCS1003003532

INTERNSHIP OFFER LETTER

Dear Tarun Kumar Yadav ,

We are pleased to offer you a 1 Month Internship as Project Intern at Codec Technologies, a global platform dedicated to empowering learners and connecting diverse talent to create meaningful career opportunities worldwide. Codec Technologies delivers dedicated IT and business consultancy services across 27+ countries, empowering global innovation and strategic growth.

Internship Details:

- Designation : Artificial Intelligence Intern
- Location & Mode: Pan India, Hybrid
- Duration : 15/06/2026 to 25/07/2026
- Reporting to : Assigned Project Head(s)

This program provides an immersive experience, enabling you to gain industry-level knowledge and acquire valuable skills. Exceptional interns may also qualify for a scholarship award based on weekly performance reviews conducted by project supervisor.

If you accept this offer and the terms above, please complete your assigned projects and training tasks on our platform and via email.

Best regards,

Sincerely,

Dr. Anurag Shrivastava,
Codec Technologies India
NCS ID: E19E86-0116588288923
Date Of Issue - 15/06/2026



Dr. Anurag Shrivastava
Talent Acquisition Manager

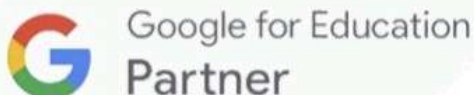


TABLE OF CONTENTS

Chapter No.	Chapter Name	Page Nos.
	Cover Page	—
1.	Candidate's Declaration	i
2.	Acknowledgement	ii
3.	Internship Completion Certificate	iii
	Table of Contents	iv
	List of Tables	v
	List of Figures	vi
4.	Project Description	1
	4.1 Introduction	1
	4.2 Organization Profile	1
	4.3 Problem Statement	2
	4.4 Project Objectives	2
	4.5 Scope of the Project	2
	4.6 Technologies and Tools Used	3
	4.7 System Architecture	3
	4.8 Methodology	5
	4.9 Expected Outcomes	7
	4.10 Certificates of Completion and Other Communication Mail Proof	8
5.	Bibliography / References	9

LIST OF TABLES

Table No.	Title	Page No.
Table 4.1	Dataset Overview (Before and After Cleaning)	2
Table 4.2	Technology Stack Summary	3
Table 4.3	Model Performance Comparison	5
Table 4.4	Ensemble Model Results	6

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 4.1	Machine Learning Pipeline Architecture	4
Figure 4.2	Class Distribution (Ham vs Spam)	4
Figure 4.3	Model Improvement Workflow	6

4. PROJECT DESCRIPTION

4.1 Introduction

SMS Spam Detection is a machine learning project developed during the internship to automatically classify SMS text messages as either “spam” (unsolicited, promotional, or fraudulent) or “ham” (legitimate). The system applies natural language processing (NLP) techniques to clean and transform raw text messages, extracts numerical features using TF-IDF vectorization, and trains multiple supervised classification algorithms to distinguish between the two categories.

The project follows a complete machine learning workflow: data cleaning, exploratory data analysis (EDA), text pre-processing, feature extraction, model building, comparative evaluation, and model improvement through ensemble techniques. The final system combines several classifiers using a soft Voting Classifier to achieve a test-set accuracy of 98.16% and a precision of 99.17%, prioritising precision so that legitimate messages are rarely misclassified as spam.

The dataset underlying the project is the SMS Spam Collection dataset, containing 5,572 labelled SMS messages compiled for spam-research purposes. Each record consists of a label (ham/spam) and the raw message text. The entire pipeline — from raw CSV to final trained model — was implemented in Python within a single Jupyter notebook (sms-spam-detection.ipynb).

4.2 Organization Profile

[Provide a short profile of the host organisation / company where the internship was undertaken here — its name, location, domain of work, and a brief history. This section should describe the organisation that assigned and supervised the SMS Spam Detection project, not the project itself.]

4.3 Problem Statement

Unsolicited spam messages are a persistent problem for SMS users, ranging from harmless promotional content to phishing and financial-fraud attempts. Manually filtering such messages is impractical at scale, and simple keyword-based filters are easily bypassed and prone to misclassifying legitimate messages. There was a need for a data-driven classification system that could learn the linguistic patterns distinguishing spam from ham messages and generalise to unseen text with high precision, since misclassifying a genuine message as spam carries a higher cost than missing an occasional spam message. The project addresses this gap by building and comparing multiple machine learning classifiers on a labelled SMS corpus and selecting the model that best balances accuracy and precision.

4.4 Project Objectives

- Clean and pre-process a raw SMS dataset into a structured, duplicate-free corpus.
- Perform exploratory data analysis to understand the statistical and linguistic differences between spam and ham messages.
- Build a text pre-processing pipeline covering lower-casing, tokenization, stop-word removal, punctuation removal, and stemming.
- Convert cleaned text into numerical features using TF-IDF vectorization.
- Train and compare eleven supervised classification algorithms on the same feature set.

- Improve model performance through feature-engineering iterations and ensemble methods (Voting and Stacking classifiers).
- Select a final model that maximises precision without materially sacrificing accuracy.

4.5 Scope of the Project

The project processes the complete SMS Spam Collection dataset of 5,572 messages. After removing 403 exact duplicate records, 5,169 unique messages remain, comprising 4,516 ham messages (87.4%) and 653 spam messages (12.6%) — a naturally imbalanced dataset that informed the choice of precision, alongside accuracy, as a primary evaluation metric. Table 4.1 summarises the dataset before and after cleaning.

Table 4.1: Dataset Overview (Before and After Cleaning)

Stage	Total Messages	Ham	Spam	Duplicate Rows
Raw dataset	5,572	—	—	403
After de-duplication	5,169	4,516 (87.4%)	653 (12.6%)	0

On average, spam messages are markedly longer than ham messages: spam messages average 137.5 characters, 27.7 words, and 3.0 sentences, compared to 70.5 characters, 17.1 words, and 1.8 sentences for ham messages — a distinction the model exploits implicitly through the TF-IDF feature space. The project's scope is limited to binary text classification of English-language SMS messages using the provided dataset; it does not include deployment as a live web service, multi-language support, or real-time message interception, which are identified as future work in Section 4.9.

4.6 Technologies and Tools Used

Table 4.2 lists the complete technology stack used across the data-cleaning, analysis, and modelling layers of the project.

Table 4.2: Technology Stack Summary

Layer	Technology	Details
Language	Python 3	Core language for the full analysis and modelling pipeline (Jupyter Notebook)
Data Handling	Pandas, NumPy	DataFrame operations, cleaning, aggregation, descriptive statistics
NLP	NLTK	Word/sentence tokenization, English stop-word list, Porter Stemmer
Feature Extraction	scikit-learn (CountVectorizer, TfidfVectorizer)	Bag-of-Words and TF-IDF vectorization, max_features = 3,000
Machine Learning	scikit-learn, XGBoost	11 classification algorithms plus Voting and Stacking ensembles
Visualization	Matplotlib, Seaborn, WordCloud	Class distribution, length distributions, correlation heat-map, spam/ham word

Layer	Technology	Details
		clouds
Environment	Jupyter Notebook	Interactive development and analysis (sms-spam-detection.ipynb)

4.7 System Architecture

The project follows a linear machine-learning pipeline architecture, shown in Figure 4.1, moving from the raw labelled dataset through cleaning, text pre-processing, feature extraction, and finally model training and evaluation.

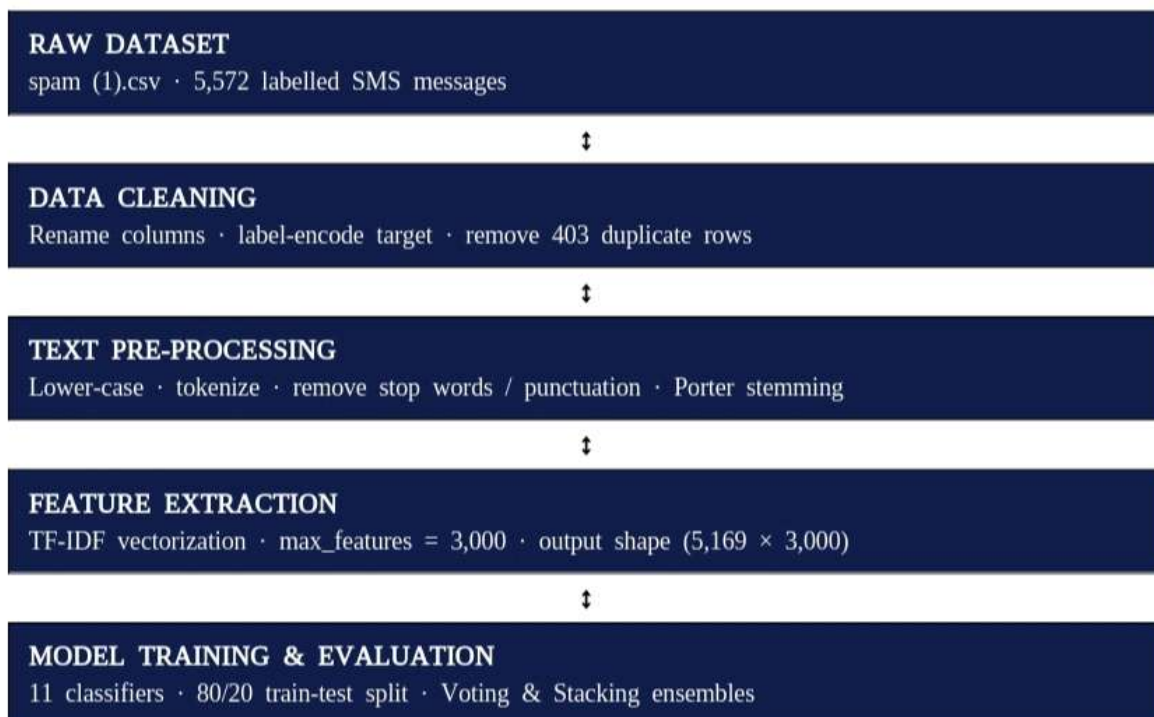


Figure 4.1: Machine Learning Pipeline Architecture

Figure 4.2 shows the class distribution of the cleaned dataset, which motivated the use of precision — not just accuracy — as a key evaluation metric throughout the project.

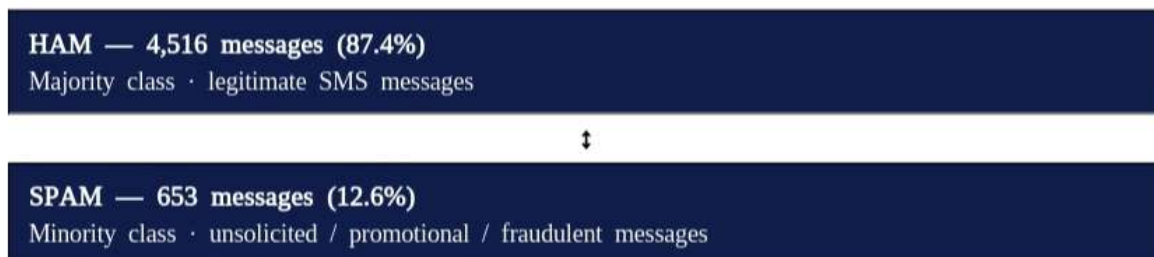


Figure 4.2: Class Distribution (Ham vs Spam)

4.7.1 Data Cleaning Module

The raw dataset (spam (1).csv) contains two primary columns, v1 (label) and v2 (message text), which are renamed to target and text respectively. The target column is label-encoded (ham = 0, spam = 1) using scikit-learn's LabelEncoder. Missing values are checked for and confirmed absent, and 403 fully duplicated rows are identified and removed using drop_duplicates(), leaving 5,169 unique records.

4.7.2 Feature Engineering Module

Three additional numeric features are derived directly from the raw text for exploratory purposes: num_characters, num_words (via NLTK's word_tokenize), and num_sentences (via NLTK's sent_tokenize). These features, together with a correlation heat-map and pair-plot, are used to confirm that spam messages are systematically longer than ham messages before any text-cleaning is applied.

4.7.3 Text Pre-processing Module

A custom transform_text() function implements the core NLP pipeline: converting text to lower case, tokenizing it with NLTK, retaining only alphanumeric tokens, removing English stop words and punctuation, and finally applying Porter stemming to each remaining token. The transformed text is stored in a new transformed_text column and used as the sole input to feature extraction. Word clouds generated separately for spam and ham messages, along with frequency counts of the 30 most common words in each class, were used to visually validate that the pre-processing pipeline was isolating meaningful, class-distinguishing vocabulary.

4.8 Methodology

4.8.1 Text Vectorization

The cleaned transformed_text column is converted into a numeric feature matrix using TfidfVectorizer with max_features capped at 3,000, producing a $(5,169 \times 3,000)$ sparse matrix. TF-IDF was retained over plain Bag-of-Words (CountVectorizer) after comparative testing showed it produced more consistent precision across classifiers. The label-encoded target column forms the target vector y, and the data is split 80/20 into training and test sets using a fixed random_state for reproducibility.

4.8.2 Model Training and Comparison

Eleven classification algorithms are trained on the identical training split and evaluated on the same held-out test set: Support Vector Classifier (sigmoid kernel), K-Nearest Neighbours, Multinomial Naive Bayes, Decision Tree, Logistic Regression, Random Forest, AdaBoost, Bagging Classifier, Extra Trees Classifier, Gradient Boosting, and XGBoost. A common train_classifier() helper function fits each model and returns its accuracy and precision on the test set, allowing all eleven results to be assembled into a single comparison table (Table 4.3) and visualised as a grouped bar chart.

Table 4.3: Model Performance Comparison (sorted by Precision)

Algorithm	Accuracy	Precision
K-Nearest Neighbours	0.9004	1.0000
Multinomial Naive Bayes	0.9594	1.0000

Algorithm	Accuracy	Precision
Extra Trees Classifier	0.9778	0.9915
Random Forest	0.9700	0.9908
SVC (sigmoid kernel)	0.9729	0.9741
AdaBoost	0.9623	0.9541
XGBoost	0.9720	0.9504
Logistic Regression	0.9516	0.9400
Gradient Boosting	0.9516	0.9314
Bagging Classifier	0.9574	0.8615
Decision Tree	0.9352	0.8381

4.8.3 Model Improvement

Multinomial Naive Bayes combined with TF-IDF (max_features = 3,000) emerged as one of the strongest simple baselines, achieving perfect precision (1.0000) on the test set — meaning zero false positives, i.e. no legitimate message was ever misclassified as spam. Because precision was prioritised (a false-positive spam label is more disruptive to a user than an occasional missed spam message), Multinomial Naive Bayes, the sigmoid-kernel SVC, and the Extra Trees Classifier were carried forward as base estimators for ensembling.

4.8.4 Ensemble Modelling

Two ensemble strategies were tested on top of the three selected base estimators (SVC, Multinomial Naive Bayes, Extra Trees Classifier), shown in Figure 4.3:



Figure 4.3: Model Improvement Workflow

Table 4.4 compares the two ensemble results against the best individual model.

Table 4.4: Ensemble Model Results

Model	Accuracy	Precision
Voting Classifier (SVC + NB + Extra Trees, soft voting)	0.9816	0.9917
Stacking Classifier (SVC + NB + Extra Trees → Random Forest)	0.9787	0.9328
Best individual model (Extra Trees Classifier)	0.9778	0.9915

The soft Voting Classifier was selected as the final model: it improved overall accuracy to 98.16% while retaining a precision of 99.17%, outperforming both the best individual classifier and the Stacking Classifier on this dataset.

4.9 Expected Outcomes

The internship delivered a complete, reproducible SMS spam classification pipeline meeting the objectives set out in Section 4.4. Key outcomes achieved include:

- A cleaned, de-duplicated dataset of 5,169 labelled SMS messages ready for modelling.
- A reusable text pre-processing function (`transform_text()`) covering tokenization, stop-word removal, and stemming.
- A comparative benchmark of eleven supervised classification algorithms evaluated on identical train/test splits.
- A final soft Voting Classifier achieving 98.16% accuracy and 99.17% precision on the held-out test set.
- Supporting exploratory visualisations, including class-distribution pie charts, character/word-count distributions, a correlation heat-map, and spam/ham word clouds.

Several enhancements were identified during the internship as scope for future iterations, rather than being implemented in the current notebook: packaging the final model with pickle/joblib for reuse; deploying the classifier behind a simple web interface, as noted in the project's own planning outline ("website" and "deploy" stages); incorporating n-gram features to capture short spam phrases; and periodically retraining the model on newer spam samples to counter concept drift.

4.10 Certificates of Completion and Other Communication Mail Proof

[Insert Internship Completion Certificate / Communication Mail Proof Here]

5. BIBLIOGRAPHY / REFERENCES

- [1] Almeida, T.A., Hidalgo, J.M.G., "SMS Spam Collection Data Set," UCI Machine Learning Repository / Kaggle.
- [2] Pedregosa, F. et al., "Scikit-learn: Machine Learning in Python," scikit-learn.org.
- [3] Bird, S., Loper, E., Klein, E., "Natural Language Toolkit (NLTK) Documentation," nltk.org.
- [4] Chen, T., Guestrin, C., "XGBoost: A Scalable Tree Boosting System," xgboost.readthedocs.io.
- [5] McKinney, W., "pandas: Python Data Analysis Library," pandas.pydata.org.
- [6] Hunter, J.D., "Matplotlib: A 2D Graphics Environment," matplotlib.org.
- [7] Waskom, M., "seaborn: Statistical Data Visualization," seaborn.pydata.org.
- [8] Mueller, A., "word_cloud: A Little Word Cloud Generator in Python," github.com/amueller/word_cloud.



CERTIFICATE OF INTERNSHIP

This certificate is awarded to
Tarun Kumar Yadav

From Codec Technologies Pvt. Ltd.

In recognition of his/her efforts and achievements in completing the 1 Month AICTE & ICAC

Approved internship program as

Artificial Intelligence Intern

Conducted from 15/06/2026 to 25/07/2026



AICTE ID - CORPORATE6759d549ce59e1733940553

Dr. Anurag Shrivastava
Program Manager